

ESP32 6 — Sending Structured Data with JSON

Tutorial Group: ESP32

Goal

Modify *ESP32 5 — Periodic Sensor Data with HTTP POST* so that the ESP32 sends several related pieces of information in one HTTP POST using JSON. The message will contain the team name, ADC value, current uptime for the ESP32 processor, and calculated voltage.

The Python server from *ESP32 4 — HTTP POST and a local Python server* is reused, but this tutorial changes it so that it parses and displays the JSON data.

What You Need

- ESP32-WROOM-32
- USB cable
- Computer with the Arduino IDE and Python 3
- Wi-Fi network

Part 1 — Why JSON?

Sending only 2476 tells the server the value, but not what the value represents. JSON allows related values to be labeled:

For comparison, a raw value might look like this: 2476

```
{
  "team": "Team 42",
  "adc": 2476,
  "voltage": 1.99,
  "uptime": 4021
}
```

JSON is widely used when computers and embedded devices exchange structured information.

A structured message can instead identify each value, for example: {"adc": 2476, "voltage": 1.99}

Part 2 — Install ArduinoJson

In the Arduino IDE, open the Library Manager and search for ArduinoJson. Install the ArduinoJson library.

Part 3 — Define the Team and Analog Input

```
const char teamName[] = "Team 42";
const int analogPin = 34;
```

Part 4 — Calculate the Voltage

Read the ADC and convert it to voltage. The ESP32 ADC used in these tutorials is treated as a 12-bit value, so its nominal range is 0–4095:

```
int adcValue = analogRead(analogPin);
float voltage = adcValue * 3.3 / 4095.0;
```

The variable `voltage` is a float because the result can contain a fractional value.

Part 5 — Build the JSON Message

```
JsonDocument doc;

doc["team"] = teamName;
doc["adc"] = adcValue;
doc["voltage"] = voltage;
doc["uptime"] = millis();

String postData;
serializeJson(doc, postData);
```

The resulting string contains the four labeled values.

The uptime value is the number of milliseconds returned by `millis()` since the ESP32 started running the current program.

Part 6 — Update the Python Server

Replace the *ESP32 4 — HTTP POST and a local Python server* with this version. Save it as `esp32_server.py` and restart the server.

```
from http.server import BaseHTTPRequestHandler, HTTPServer
import json

class ESP32Handler(BaseHTTPRequestHandler):

    def do_POST(self):
        length = int(self.headers.get('Content-Length', 0))
        data = self.rfile.read(length)

        try:
            message = json.loads(data.decode("utf-8"))

            print()
            print("POST received:")
            print("Team:    ", message["team"])
            print("ADC:      ", message["adc"])
            print("Voltage: ", message["voltage"])
            print("Uptime:  ", message["uptime"])

            self.send_response(200)

        except Exception as e:
            print("Invalid JSON:", e)
            self.send_response(400)

        self.end_headers()
        self.wfile.write(b"Received")

server = HTTPServer(("0.0.0.0", 5000), ESP32Handler)

print("ESP32 JSON server running")
print("Listening on port 5000")
print("Press Ctrl+C to stop")
```

```
server.serve_forever()
```

The `json.loads()` function converts the received JSON text into Python data that the server can access by field name.

Part 7 — Complete ESP32 Program

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>

const char* ssid = "YourNetworkName";
const char* password = "YourPassword";
const char* serverURL = "http://192.168.1.50:5000/";

const char teamName[] = "Team 42";
const int analogPin = 34;

unsigned long lastPostTime = 0;
const unsigned long postInterval = 10000;

void setup()
{
    Serial.begin(115200);

    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED)
    {
        Serial.print(".");
        delay(500);
    }

    Serial.println(" Wi-Fi connected");
}

void loop()
{
    if ((millis() - lastPostTime) >= postInterval)
    {
        lastPostTime = millis();

        int adcValue = analogRead(analogPin);
        float voltage = adcValue * 3.3 / 4095.0;

        JsonDocument doc;

        doc["team"] = teamName;
        doc["adc"] = adcValue;
        doc["voltage"] = voltage;
        doc["uptime"] = millis();

        String postData;
        serializeJson(doc, postData);

        HTTPClient http;
        http.begin(serverURL);
```

```

        http.addHeader("Content-Type", "application/json");

        int responseCode = http.POST(postData);
        Serial.println(postData);
        Serial.print("HTTP Response: ");
        Serial.println(responseCode);

        http.end();
    }
}

```

Part 8 — Observe the Results

The Python server should display information similar to:

```

POST received:
Team:      Team 42
ADC:       2476
Voltage:   1.99
Uptime:    4021

```

Part 9 — Experiment

- Add another field to the JSON message.
- Change the team name.
- Send the voltage with more or fewer decimal places and observe the result.
- Change the uptime value and observe how the server displays it. The complete program already includes uptime as milliseconds returned by `millis()`.

What You Should Understand

- Why structured data is useful when multiple related values are transmitted.
- How to create a JSON message on the ESP32.
- How to send JSON in an HTTP POST.
- How Python can parse JSON and access individual fields.
- Why the calculated voltage is stored as a float.

Before Moving On

You should be able to answer **yes** to all of the following:

- ☐ I understand why structured data is useful.
- ☐ I can create a JSON message on the ESP32.
- ☐ I can send JSON in an HTTP POST.
- ☐ I can parse JSON in Python.
- ☐ I can access individual JSON fields on the server.
- ☐ I understand why voltage is stored as a float.